

OpenBin 2.0

Complete PostgreSQL & Firebase Database Design

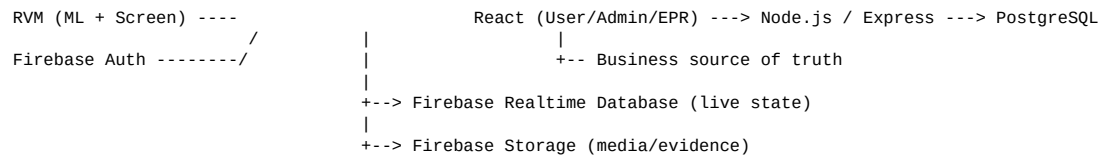
Corrected full version — rebuilt from the finalized OpenBin architecture.

Final stack: PostgreSQL for business data + Firebase Authentication + Firebase Realtime Database + Firebase Storage. MongoDB and Firestore are excluded from the current architecture.

This document defines the database ownership, entities, relationships, transaction lifecycle, QR claim lifecycle, reward/charity accounting, collection and EPR traceability, Firebase structures, security boundaries, and implementation order.

1. Final Architecture

Node.js is the central application/API layer. React and the RVM do not directly access PostgreSQL. PostgreSQL is the authoritative source for business records. Firebase provides identity, live state and file storage.



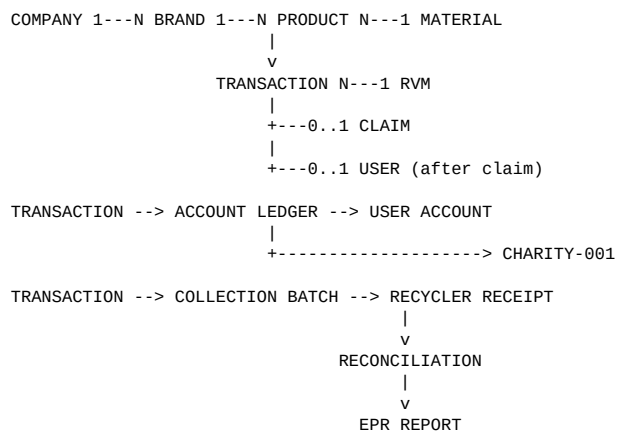
Technology ownership

Component	Responsibility	Authoritative?
PostgreSQL	Users, RVMs, transactions, QR claims, accounts, ledger, collection, reconciliation, EPR, audit	YES — business source of truth
Firebase Authentication	Login and Firebase UID	Identity provider
Firebase Realtime Database	RVM online/processing/live dashboard state	NO — current state only
Firebase Storage	Images/videos/evidence files	File store
Node.js + Express	Validation, authorization, business logic, DB transactions	Application control layer
React	User/Admin/EPR UI	NO

2. Core Design Principles

1. One source of truth for business transactions: PostgreSQL.
2. Authentication identity and business identity are linked but not duplicated: Firebase UID → users.firebase_uid.
3. RVM transaction and user reward claim are separate events.
4. A successfully accepted bottle creates an RVM transaction even if the QR is never scanned.
5. An expired, unclaimed QR transfers its reward to the single OpenBin Charity Account.
6. User donations and automatic unclaimed-reward transfers both credit the same CHARITY-001 account but remain distinguishable in the ledger.
7. Financial/reward history is append-oriented; corrections use reversals/adjustments, not deletion.
8. EPR reporting should distinguish OpenBin-recorded quantities from quantities that have passed the project's collection/reconciliation/verification process.
9. Realtime Firebase data is not a historical transaction store.
10. Critical operations are performed in PostgreSQL transactions with idempotency and concurrency protection.

3. Domain Relationships



4. PostgreSQL Table Inventory

Domain	Table	Purpose
Identity	users	Firebase UID mapping, application role/status
RVM	rvms	Registered reverse vending machines
Catalog	companies	EPR/company organization
Catalog	brands	Company-owned brands
Catalog	materials	Supported recyclable materials
Catalog	products	Brand/product/material mapping
Pricing	material_prices	Historical reward/material rates
Transactions	transactions	Authoritative RVM bottle events
Transactions	transaction_claims	QR claim lifecycle
Accounts	accounts	User accounts + single CHARITY-001
Accounts	account_transactions	Append-only ledger
Accounts	withdrawals	Withdrawal lifecycle
Accounts	donations	Explicit user donation records
Collection	collection_batches	Physical collection batches
Collection	collection_batch_rvms	RVM-to-batch attribution
Collection	recyclers	Recycler/facility records
Collection	recycler_receipts	Receipt/evidence from recycler
Collection	reconciliations	Recorded vs collected vs received
EPR	epr_reports	Reporting-period snapshots
EPR	epr_report_batches	Report-to-batch links
Audit	audit_logs	Security/business audit trail

5. Identity & Catalog Tables

users

Application user linked to Firebase Authentication.

Column	Type	Null?	Key / default	Description
user_id	UUID	NO	PK	Internal OpenBin user ID.
firebase_uid	VARCHAR(128)	NO	UNIQUE	Firebase Authentication UID.
full_name	VARCHAR(150)	NO		Application name.
email	VARCHAR(255)	NO	UNIQUE	Application email.
phone	VARCHAR(30)	YES		Optional contact.
role	user_role	NO	USER	USER / ADMIN / EPR.
status	user_status	NO	ACTIVE	ACTIVE / BLOCKED / SUSPENDED.
created_at	TIMESTAMPTZ	NO	now()	Created time.
updated_at	TIMESTAMPTZ	NO	now()	Updated time.

Constraints: UNIQUE(firebase_uid), UNIQUE(email).

Notes: Firebase owns authentication; PostgreSQL owns OpenBin authorization/application data.

rvms

Registered RVM/device.

Column	Type	Null?	Key / default	Description
rvm_id	UUID	NO	PK	Internal RVM ID.
rvm_code	VARCHAR(50)	NO	UNIQUE	RVM-001 style code.
serial_number	VARCHAR(100)	NO	UNIQUE	Device serial.
location_name	VARCHAR(200)	YES		Installation location.
latitude	NUMERIC(9,6)	YES		Optional coordinate.
longitude	NUMERIC(9,6)	YES		Optional coordinate.
status	rvm_status	NO	ACTIVE	ACTIVE / MAINTENANCE / DISABLED.
installed_at	TIMESTAMPTZ	YES		Installation time.
last_seen_at	TIMESTAMPTZ	YES		Last backend contact.
created_at	TIMESTAMPTZ	NO	now()	Created time.
updated_at	TIMESTAMPTZ	NO	now()	Updated time.

Constraints: UNIQUE(rvm_code), UNIQUE(serial_number).

Notes: RVM credentials/secrets are not exposed to React.

companies

Company/EPR entity.

Column	Type	Null?	Key / default	Description
company_id	UUID	NO	PK	Company ID.
company_name	VARCHAR(200)	NO		Name.
registration_number	VARCHAR(100)	YES	UNIQUE if applicable	Business/regulatory reference.
email	VARCHAR(255)	YES		Contact.
phone	VARCHAR(30)	YES		Contact.
address	TEXT	YES		Address.
status	entity_status	NO	ACTIVE	ACTIVE / INACTIVE.
created_at	TIMESTAMPTZ	NO	now()	Created time.
updated_at	TIMESTAMPTZ	NO	now()	Updated time.

brands

Brand owned by a company.

Column	Type	Null?	Key / default	Description
brand_id	UUID	NO	PK	Brand ID.
company_id	UUID	NO	FK → companies	Owner.
brand_name	VARCHAR(200)	NO		Brand name.

Column	Type	Null?	Key / default	Description
status	entity_status	NO	ACTIVE	ACTIVE / INACTIVE.
created_at	TIMESTAMPTZ	NO	now()	Created time.
updated_at	TIMESTAMPTZ	NO	now()	Updated time.

Constraints: UNIQUE(company_id, brand_name).

materials

Material categories.

Column	Type	Null?	Key / default	Description
material_id	UUID	NO	PK	Material ID.
material_code	VARCHAR(30)	NO	UNIQUE	PET, AL, etc.
material_name	VARCHAR(100)	NO		Display name.
unit	VARCHAR(20)	NO	kg	Reporting unit.
status	entity_status	NO	ACTIVE	ACTIVE / INACTIVE.
created_at	TIMESTAMPTZ	NO	now()	Created time.

products

Product mapping.

Column	Type	Null?	Key / default	Description
product_id	UUID	NO	PK	Product ID.
brand_id	UUID	NO	FK → brands	Brand.
material_id	UUID	NO	FK → materials	Primary material.
product_name	VARCHAR(200)	NO		Product/package name.
volume_ml	INTEGER	YES		Optional package volume.
status	entity_status	NO	ACTIVE	ACTIVE / INACTIVE.
created_at	TIMESTAMPTZ	NO	now()	Created time.
updated_at	TIMESTAMPTZ	NO	now()	Updated time.

6. Pricing & RVM Transaction Tables

material_prices

Historical reward/material rates.

Column	Type	Null?	Key / default	Description
price_id	UUID	NO	PK	Rate ID.
material_id	UUID	NO	FK → materials	Material.
price_per_kg	NUMERIC(12,4)	NO		Rate.
effective_from	TIMESTAMPTZ	NO		Start.
effective_to	TIMESTAMPTZ	YES		End.
created_by	UUID	NO	FK → users	Admin.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Constraints: Do not allow overlapping effective periods for the same material.

Notes: Store the final reward value on each transaction so future rate changes do not rewrite history.

transactions

Authoritative record of an accepted/rejected RVM bottle event.

Column	Type	Null?	Key / default	Description
transaction_id	UUID	NO	PK	Unique transaction.
device_event_id	VARCHAR(120)	NO	UNIQUE	RVM-generated idempotency key.
rvm_id	UUID	NO	FK → rvms	Source RVM.
product_id	UUID	YES	FK → products	Identified product, if known.
material_id	UUID	NO	FK → materials	Accepted/detected material.
brand_id	UUID	YES	FK → brands	Identified brand, if known.
estimated_weight_g	NUMERIC(10,3)	NO		Estimated weight.
ml_confidence	NUMERIC(5,4)	YES		0..1 if available.
reward_value	NUMERIC(12,2)	NO		Reward assigned at acceptance.
reward_unit	VARCHAR(20)	NO	POINTS	Explicit unit.
transaction_status	transaction_status	NO	ACCEPTED	PROCESSING / ACCEPTED / REJECTED / FAILED.
claim_status	claim_status	NO	UNCLAIMED	UNCLAIMED / CLAIMED / EXPIRED / NOT_APPLICABLE.
user_id	UUID	YES	FK → users	Filled after successful claim.
created_at	TIMESTAMPTZ	NO	now()	RVM event time.
expires_at	TIMESTAMPTZ	YES		QR claim deadline.
claimed_at	TIMESTAMPTZ	YES		Successful claim time.

Constraints: UNIQUE(device_event_id); weight >= 0; reward >= 0.

Notes: Important: an accepted transaction exists even when the QR is never scanned. The claim is a separate lifecycle.

transaction_claims

QR claim record.

Column	Type	Null?	Key / default	Description
claim_id	UUID	NO	PK	Claim ID.
transaction_id	UUID	NO	UNIQUE FK	One successful claim max.
user_id	UUID	NO	FK → users	Claiming user.
qr_token_hash	VARCHAR(128)	NO	UNIQUE	Hash of claim token.
status	VARCHAR(20)	NO	CLAIMED	CLAIMED / REJECTED / EXPIRED.
generated_at	TIMESTAMPTZ	NO		QR creation.
claimed_at	TIMESTAMPTZ	YES		Claim time.
expires_at	TIMESTAMPTZ	NO		Expiry.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Constraints: UNIQUE(transaction_id), UNIQUE(qr_token_hash).

Notes: Persist the token hash, not a reusable raw claim secret. Use row locking + unique constraints to prevent double claims.

7. Accounts, Rewards & Charity

accounts

Unified user/charity account model.

Column	Type	Null?	Key / default	Description
account_id	UUID	NO	PK	Account ID.
account_type	account_type	NO		USER or CHARITY.
user_id	UUID	YES	UNIQUE FK → users	Owner for USER account.
account_code	VARCHAR(50)	NO	UNIQUE	USER-... or CHARITY-001.
balance	NUMERIC(14,2)	NO	0	Cached balance.
currency	CHAR(3)	NO	INR	Currency.
status	account_status	NO	ACTIVE	ACTIVE / FROZEN / CLOSED.
created_at	TIMESTAMPTZ	NO	now()	Created time.
updated_at	TIMESTAMPTZ	NO	now()	Updated time.

Constraints: Exactly one active CHARITY account: CHARITY-001. USER accounts have one unique user_id.

Notes: The ledger is authoritative for history; balance is a cached/current value maintained atomically.

account_transactions

Append-oriented account ledger.

Column	Type	Null?	Key / default	Description
account_transaction_id	UUID	NO	PK	Ledger entry.
account_id	UUID	NO	FK → accounts	Affected account.
source_type	account_source_type	NO		RVM_REWARD / USER_DONATION / WITHDRAWAL / CHARITY_TRANSFER / ADJUSTMENT.
source_id	UUID	YES		Origin record.
direction	ledger_direction	NO		CREDIT / DEBIT.
amount	NUMERIC(14,2)	NO		Positive magnitude.
description	TEXT	YES		Explanation.
created_at	TIMESTAMPTZ	NO	now()	Ledger time.

Constraints: CHECK(amount > 0). Never delete ledger entries; use reversals/adjustments.

Notes: Expired QR reward credits CHARITY-001. User donation debits user + credits CHARITY-001.

withdrawals

User withdrawal lifecycle.

Column	Type	Null?	Key / default	Description
withdrawal_id	UUID	NO	PK	Withdrawal ID.
account_id	UUID	NO	FK → accounts	Must be USER account.
account_transaction_id	UUID	YES	FK → account_transactions	Debit reference.
amount	NUMERIC(14,2)	NO		Amount.
method	VARCHAR(40)	NO		Withdrawal method.
destination_reference	VARCHAR(255)	YES		Masked/reference only.
status	withdrawal_status	NO	REQUESTED	REQUESTED / PROCESSING / COMPLETED / FAILED / CANCELLED.
requested_at	TIMESTAMPTZ	NO	now()	Requested time.
processed_at	TIMESTAMPTZ	YES		Processing completion.

Constraints: Handle external callbacks idempotently.

donations

Explicit user donation to the one charity account.

Column	Type	Null?	Key / default	Description
donation_id	UUID	NO	PK	Donation ID.
donor_account_id	UUID	NO	FK → accounts	User account.

Column	Type	Null?	Key / default	Description
charity_account_id	UUID	NO	FK → accounts	Must be CHARITY-001.
account_transaction_id	UUID	YES	FK → account_transactions	Ledger reference.
amount	NUMERIC(14,2)	NO		Donation amount.
status	donation_status	NO	COMPLETED	REQUESTED / COMPLETED / FAILED / REVERSED.
created_at	TIMESTAMPTZ	NO	now()	Donation time.

Constraints: CHECK(amount > 0). No multiple charity-target table is required.

8. Collection & Recycler Tables

collection_batches

Physical collection batch.

Column	Type	Null?	Key / default	Description
collection_batch_id	UUID	NO	PK	Batch ID.
batch_code	VARCHAR(60)	NO	UNIQUE	Human-readable batch code.
material_id	UUID	NO	FK → materials	Material.
recorded_weight_kg	NUMERIC(14,3)	NO		Collected weight.
collector_reference	VARCHAR(120)	YES		Collector/route reference.
collected_at	TIMESTAMPTZ	NO		Collection time.
status	collection_status	NO	RECORDED	RECORDED / IN_TRANSIT / DELIVERED / VERIFIED / FLAGGED.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Constraints: CHECK(recorded_weight_kg >= 0).

collection_batch_rvms

RVM contribution to a collection batch.

Column	Type	Null?	Key / default	Description
collection_batch_id	UUID	NO	FK → collection_batches	Batch.
rvm_id	UUID	NO	FK → rvms	RVM.
recorded_quantity_kg	NUMERIC(14,3)	YES		Optional RVM-attributed quantity.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Constraints: PRIMARY KEY(collection_batch_id, rvm_id).

recyclers

Recycler/facility entity.

Column	Type	Null?	Key / default	Description
recycler_id	UUID	NO	PK	Recycler ID.
recycler_name	VARCHAR(200)	NO		Name.
registration_number	VARCHAR(100)	YES	UNIQUE if applicable	Registration reference.
contact_email	VARCHAR(255)	YES		Contact.
contact_phone	VARCHAR(30)	YES		Contact.
address	TEXT	YES		Facility address.
status	entity_status	NO	ACTIVE	ACTIVE / INACTIVE.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Notes: Regulatory eligibility/authorization must be validated for the applicable jurisdiction before treating a recycler as an EPR-verifying party.

recycler_receipts

Evidence that a recycler received material.

Column	Type	Null?	Key / default	Description
receipt_id	UUID	NO	PK	Receipt ID.
collection_batch_id	UUID	NO	FK → collection_batches	Batch.
recycler_id	UUID	NO	FK → recyclers	Receiving recycler.
received_weight_kg	NUMERIC(14,3)	NO		Received weight.
received_at	TIMESTAMPTZ	NO		Receipt time.
document_reference	VARCHAR(255)	YES		Storage/document reference.
verification_status	verification_status	NO	PENDING	PENDING / VERIFIED / REJECTED.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Constraints: CHECK(received_weight_kg >= 0).

reconciliations

Quantity comparison and verification result.

Column	Type	Null?	Key / default	Description
reconciliation_id	UUID	NO	PK	Reconciliation ID.
collection_batch_id	UUID	NO	UNIQUE FK	Batch.
recorded_quantity_kg	NUMERIC(14,3)	NO		From OpenBin records.
collected_quantity_kg	NUMERIC(14,3)	NO		Physical collection.
received_quantity_kg	NUMERIC(14,3)	NO		Recycler received.
variance_kg	NUMERIC(14,3)	NO		Calculated variance.
tolerance_kg	NUMERIC(14,3)	NO		Accepted tolerance.
status	reconciliation_status	NO	PENDING	PENDING / RECONCILED / FLAGGED / REJECTED.
verified_by	UUID	YES	FK → users	Verifier.
verified_at	TIMESTAMPTZ	YES		Verification time.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Constraints: Quantities must be non-negative. Backend calculates variance/status.

9. EPR & Audit Tables

epr_reports

Reporting-period snapshot for a company.

Column	Type	Null?	Key / default	Description
report_id	UUID	NO	PK	Report ID.
company_id	UUID	NO	FK → companies	Reporting company.
period_start	DATE	NO		Period start.
period_end	DATE	NO		Period end.
total_transactions	BIGINT	NO	0	Relevant count.
total_recorded_weight_kg	NUMERIC(16,3)	NO	0	Recorded quantity.
total_verified_weight_kg	NUMERIC(16,3)	NO	0	Reconciled/verified quantity.
status	report_status	NO	DRAFT	DRAFT / FINAL / SUBMITTED / AMENDED.
generated_by	UUID	NO	FK → users	Generator.
generated_at	TIMESTAMPTZ	NO	now()	Generation time.

Constraints: Unique reporting period unless amendments/versioning are required.

Notes: Exact regulatory fields must be aligned with the applicable EPR framework before production use.

epr_report_batches

Report-to-collection-batch link.

Column	Type	Null?	Key / default	Description
report_id	UUID	NO	FK → epr_reports	Report.
collection_batch_id	UUID	NO	FK → collection_batches	Supporting batch.
included_weight_kg	NUMERIC(14,3)	NO		Weight attributed to report.
created_at	TIMESTAMPTZ	NO	now()	Created time.

Constraints: PRIMARY KEY(report_id, collection_batch_id).

audit_logs

Append-oriented audit trail.

Column	Type	Null?	Key / default	Description
audit_id	UUID	NO	PK	Audit event.
actor_user_id	UUID	YES	FK → users	Human actor if any.
actor_type	VARCHAR(30)	NO		USER / ADMIN / EPR / SYSTEM / RVM.
action	VARCHAR(100)	NO		Business/security action.
entity_type	VARCHAR(80)	NO		Affected entity.
entity_id	UUID	YES		Affected record.
before_data	JSONB	YES		Sanitized before snapshot.
after_data	JSONB	YES		Sanitized after snapshot.
request_id	VARCHAR(100)	YES		Correlation ID.
ip_address	INET	YES		If appropriate.
created_at	TIMESTAMPTZ	NO	now()	Audit time.

Notes: Never store passwords, private keys, payment credentials or other secrets in audit snapshots.

10. Firebase Authentication Design

Firebase Authentication is the identity provider. It should not be used as the business database.

Firestore field	Use	PostgreSQL mapping
uid	Stable identity	users.firebase_uid UNIQUE
email	Login identity	users.email
displayName	Optional profile display	users.full_name can be synchronized
photoURL	Optional profile image	Optional application reference
providerData	Auth provider metadata	Firebase-owned
custom claims	Optional coarse role hint	Should mirror DB authorization; not the sole authority

Authentication flow

```
React -> Firebase Auth -> Firebase ID token
      |
      v
    Node.js API
      |
    verify ID token signature
      |
      v
PostgreSQL users.firebase_uid
      |
      v
    authorize operation
```

Firebase UID is not the same thing as an OpenBin user ID. Keep the internal UUID in PostgreSQL and map it to Firebase UID.

11. Firebase Realtime Database Design

Use Realtime Database only for live/ephemeral state: online status, current RVM state, progress and system status. Permanent transaction history remains in PostgreSQL.

```
{
  "rvm_status": {
    "RVM-001": {
      "status": "ONLINE",
      "current_state": "IDLE",
      "current_transaction_id": null,
      "last_seen_at": "2026-09-19T10:30:00Z",
      "firmware_version": "1.0.3"
    }
  },
  "processing_status": {
    "RVM-001": {
      "state": "PROCESSING",
      "transaction_id": "TX-...",
      "progress": 0.70,
      "updated_at": "2026-09-19T10:30:10Z"
    }
  },
  "system_status": {
    "maintenance_mode": false,
    "updated_at": "2026-09-19T10:30:00Z"
  }
}
```

Path	Client read	Write	Rule
rvm_status/	Authenticated dashboard as needed	Backend/trusted device	Live state only
processing_status/	Authenticated dashboard as needed	Backend	Never the historical transaction store
system_status	Authenticated clients as needed	Admin/backend	Admin-controlled

12. Firebase Storage Design

If the project retains RVM images/video/evidence, store the files in Firebase Storage and keep only metadata/references in PostgreSQL. If raw media is not needed for the deployed product, Storage can be omitted from the MVP.

```
/rvm-evidence/  
  /<rvm_code>/  
    /<transaction_id>/  
      input/  
        frame.jpg  
        clip.mp4  
      output/  
        detection.jpg  
        result.json
```

Metadata to keep in PostgreSQL	Why
transaction_id	Links evidence to the business event
rvm_id	Links evidence to the device
storage_path / object key	Locates the file
file_type	Image/video/result JSON
uploaded_at	Audit/timing
sha256/hash (optional)	Detects accidental file changes
retention_status	Supports deletion/retention policy

13. Firebase Security Boundary

Resource	Client access	Backend access
Firebase Auth	Sign-in	Token verification/Admin SDK
Realtime rvm_status	Read as required	Write
Realtime processing_status	Read as required	Write
Storage evidence	Controlled read	Upload/read
PostgreSQL	NO direct access	Node.js only

Do not put PostgreSQL credentials or Firebase Admin private keys in React or RVM firmware. Use backend secret management and TLS.

14. Complete RVM → Database Flow

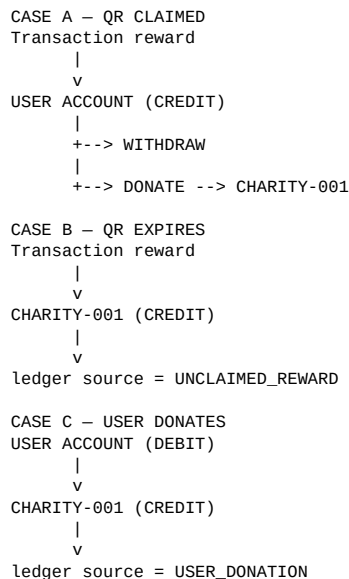
1. Bottle inserted.
2. RVM camera/ML validates bottle.
3. If rejected: record a rejected event if required; no reward QR.
4. If accepted: create unique device_event_id.
5. RVM → Node.js authenticated API.
6. Node.js validates RVM and payload.
7. PostgreSQL INSERT transactions (ACCEPTED, UNCLAIMED).
8. Backend creates short-lived claim token; store only token hash.
9. RVM displays QR.
10. Firebase Realtime Database may show PROCESSING/IDLE status for dashboards.
11. User scans QR.
12. Firebase Auth signs/authenticates user.
13. React sends Firebase ID token + claim token to Node.js.
14. Node.js verifies Firebase token and maps UID to PostgreSQL user.
15. PostgreSQL atomically claims transaction and credits user account.

15. QR Not Scanned Flow

```
Accepted bottle
|
v
transactions row exists
claim_status = UNCLAIMED
|
v
QR displayed
|
| no scan
v
expires_at reached
|
v
backend expiry job locks transaction
|
+--> if still UNCLAIMED and ACCEPTED:
      mark EXPIRED
      credit CHARITY-001 exactly once
      write audit event
|
v
original RVM transaction remains in PostgreSQL
```

Therefore: unscanned QR = valid RVM transaction, but no user claim. The reward is transferred to the single charity account after expiry.

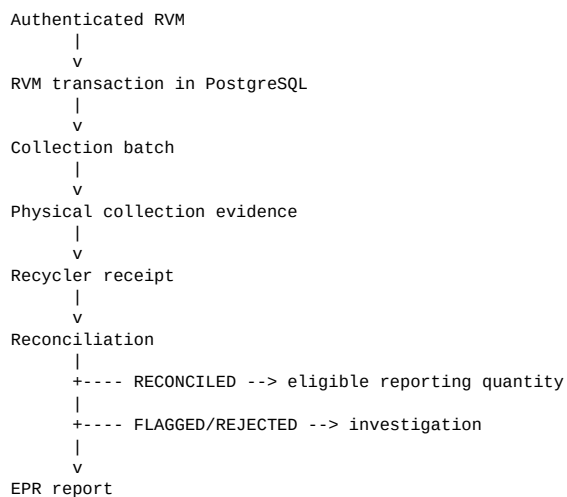
16. Reward & Account Flows



One charity account

There is exactly one system charity account: **CHARITY-001**. No charity-target table is needed for the current design. The ledger distinguishes why money entered the account: `USER_DONATION` versus `UNCLAIMED_REWARD`.

17. Data Trust / EPR Traceability



PostgreSQL consistency is not by itself proof that a physical recycling event occurred. OpenBin's trust model should combine authenticated devices, immutable/append-oriented records, collection evidence, recycler receipts, reconciliation and any external verification required by the applicable EPR process.

18. PostgreSQL DDL Blueprint

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TYPE user_role AS ENUM ('USER', 'ADMIN', 'EPR');
CREATE TYPE user_status AS ENUM ('ACTIVE', 'BLOCKED', 'SUSPENDED');
CREATE TYPE rvm_status AS ENUM ('ACTIVE', 'MAINTENANCE', 'DISABLED');
CREATE TYPE entity_status AS ENUM ('ACTIVE', 'INACTIVE');
CREATE TYPE transaction_status AS ENUM
('PROCESSING', 'ACCEPTED', 'REJECTED', 'FAILED');
CREATE TYPE claim_status AS ENUM
('UNCLAIMED', 'CLAIMED', 'EXPIRED', 'NOT_APPLICABLE');
CREATE TYPE account_type AS ENUM ('USER', 'CHARITY');
CREATE TYPE account_status AS ENUM ('ACTIVE', 'FROZEN', 'CLOSED');
CREATE TYPE ledger_direction AS ENUM ('CREDIT', 'DEBIT');

CREATE TABLE users (
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  firebase_uid VARCHAR(128) NOT NULL UNIQUE,
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  role user_role NOT NULL DEFAULT 'USER',
  status user_status NOT NULL DEFAULT 'ACTIVE',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE rvms (
  rvm_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  rvm_code VARCHAR(50) NOT NULL UNIQUE,
  serial_number VARCHAR(100) NOT NULL UNIQUE,
  status rvm_status NOT NULL DEFAULT 'ACTIVE',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE transactions (
  transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  device_event_id VARCHAR(120) NOT NULL UNIQUE,
  rvm_id UUID NOT NULL REFERENCES rvms(rvm_id),
  material_id UUID NOT NULL REFERENCES materials(material_id),
  product_id UUID REFERENCES products(product_id),
  user_id UUID REFERENCES users(user_id),
  estimated_weight_g NUMERIC(10,3) NOT NULL CHECK (estimated_weight_g >= 0),
  reward_value NUMERIC(12,2) NOT NULL CHECK (reward_value >= 0),
  transaction_status transaction_status NOT NULL DEFAULT 'ACCEPTED',
  claim_status claim_status NOT NULL DEFAULT 'UNCLAIMED',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ,
  claimed_at TIMESTAMPTZ
);

CREATE TABLE accounts (
  account_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_type account_type NOT NULL,
  user_id UUID UNIQUE REFERENCES users(user_id),
  account_code VARCHAR(50) NOT NULL UNIQUE,
  balance NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (balance >= 0),
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  status account_status NOT NULL DEFAULT 'ACTIVE'
);

CREATE TABLE account_transactions (
  account_transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id UUID NOT NULL REFERENCES accounts(account_id),
  source_id UUID,
  direction ledger_direction NOT NULL,
  amount NUMERIC(14,2) NOT NULL CHECK (amount > 0),
  description TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Seed exactly one system charity account:
-- account_code = 'CHARITY-001', account_type = 'CHARITY'
```

The DDL above is a blueprint, not a full migration. The complete table definitions in sections 5–9 should be implemented as versioned migrations.

19. Indexes, Idempotency & Concurrency

```
CREATE INDEX idx_transactions_rvm_created
  ON transactions(rvm_id, created_at DESC);

CREATE INDEX idx_transactions_user_created
  ON transactions(user_id, created_at DESC);

CREATE INDEX idx_transactions_status
  ON transactions(transaction_status, claim_status);

CREATE INDEX idx_claims_user
  ON transaction_claims(user_id, claimed_at DESC);

CREATE INDEX idx_account_transactions_account
  ON account_transactions(account_id, created_at DESC);

CREATE INDEX idx_collection_batches_material_date
  ON collection_batches(material_id, collected_at DESC);

CREATE INDEX idx_epr_reports_company_period
  ON epr_reports(company_id, period_start, period_end);

CREATE INDEX idx_audit_entity
  ON audit_logs(entity_type, entity_id, created_at DESC);
```

Atomic successful claim

```
BEGIN;
SELECT * FROM transactions
WHERE transaction_id = $1
FOR UPDATE;

-- Verify:
-- transaction_status = ACCEPTED
-- claim_status = UNCLAIMED
-- now() < expires_at
-- token hash matches

UPDATE transactions
SET user_id = $user_id,
    claim_status = 'CLAIMED',
    claimed_at = now()
WHERE transaction_id = $transaction_id;

INSERT INTO account_transactions
(account_id, source_id, direction, amount, description)
VALUES
($user_account, $transaction_id, 'CREDIT', $reward,
 'RVM reward claim');

-- Update cached balance + audit in the same DB transaction.
COMMIT;
```

Atomic expired QR → charity

```
BEGIN;
SELECT * FROM transactions
WHERE transaction_id = $1
FOR UPDATE;

-- Verify ACCEPTED + UNCLAIMED + expired.

UPDATE transactions
SET claim_status = 'EXPIRED'
WHERE transaction_id = $transaction_id
  AND claim_status = 'UNCLAIMED';

-- Only if the UPDATE changed one row:
INSERT INTO account_transactions
(account_id, source_id, direction, amount, description)
VALUES
($charity_account, $transaction_id, 'CREDIT', $reward,
 'Unclaimed reward after QR expiry');

-- Update cached charity balance + audit.
COMMIT;
```

The expiry operation must be idempotent. A second worker run must not create a second charity credit.

20. Implementation Roadmap

Phase	Work	Deliverable
1	Create PostgreSQL database + pgcrypto	Working database
2	Create enums + core entities	users, rvms, companies, brands, materials, products
3	Create transaction + claim	RVM lifecycle + QR lifecycle
4	Create accounts + ledger	User accounts + CHARITY-001
5	Create collection + recycler + reconciliation	Physical traceability
6	Create EPR + audit	Reporting and audit
7	Add indexes + constraints	Performance/integrity
8	Create Firebase project	Auth + Realtime DB + Storage
9	Build Node.js DB layer	Pool + migrations + transactions
10	RVM transaction API	RVM → Node.js → PostgreSQL
11	QR claim API	Firebase Auth → Node.js → PostgreSQL
12	Expiry worker	Expired claims → CHARITY-001 exactly once
13	Collection/EPR APIs	Traceable reports
14	Security/concurrency tests	Idempotency, authorization, race-condition tests

21. Final Locked Decisions

Decision	Final choice
Primary DB	PostgreSQL
Business source of truth	PostgreSQL
Authentication	Firebase Authentication
Realtime/live state	Firebase Realtime Database
Media/evidence	Firebase Storage
API	Node.js + Express
Frontend	React
MongoDB	Removed
Firestore	Not used in current architecture
Blockchain	Not required in current architecture
Accepted + unscanned QR	Valid RVM transaction; expires; reward → CHARITY-001
Charity accounts	Exactly one system account: CHARITY-001
User reward	User account → Withdraw or Donate
EPR traceability	Transaction → Collection → Recycler Receipt → Reconciliation → EPR Report

22. Production Cautions

- Use migrations, not manual production edits.
- Never expose DB credentials to clients.
- Use TLS.
- Hash QR claim tokens.
- Use idempotency keys for RVM events and external callbacks.
- Lock rows/transactions for reward claims and account movements.
- Never delete ledger history.
- Define media retention/privacy rules.
- Validate the exact EPR reporting and verification requirements for the jurisdiction before calling a quantity externally verified.

End of corrected full database design — OpenBin 2.0